Connected Autonomous Vehicle Platoon Control Considering Vehicle Dynamic Information

Introduction

This study proposes a platoon control protocol for connected autonomous vehicles traveling on lane-disciplined roads within a vehicle-to-vehicle (V2V) communication environment, taking into account the influence of vehicle dynamic information. Directed graph theory is employed to model the communication topology among vehicles. Additionally, the protocol integrates electronic throttle angle, which encapsulates the vehicle's dynamic characteristics. The stability and consensus of the proposed control strategy are analyzed using the Routh–Hurwitz stability criterion. Furthermore, the protocol is designed to ensure that vehicles achieve consensus on spacing error and velocity. Finally, the effectiveness of the control approach is validated through numerical simulations. The remainder of the document is organized as follows: Section 2 describes the mathematical formulation and background; Section 3 details the control protocol; Section 4 comments on convergence properties; Section 5 illustrates numerical experiments; finally, Section 6 concludes the study.

2. Formulation and Preliminaries

2.1 Communication Topology and Graph Theory

The figure below illustrates the communication topology used in the platoon system. The leader only communicates with the first follower. Each follower receives information from the vehicle immediately ahead, forming a directed line topology. This structure ensures simplicity and scalability while maintaining the desired string stability in vehicle platoons.

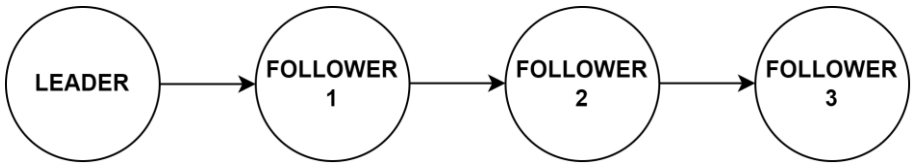


Figure: Communication Topology

This study adopts a common communication topology where connected autonomous vehicles (CAVs) share velocity, position data via V2V communication. The platoon structure consists of one leader vehicle and multiple followers. The leader moves with an initial state and desired velocity, which followers aim to track to maintain safe gaps and velocity alignment.

Graph theory is employed to model these communication links. Each vehicle is a node in a directed graph where edges represent data exchange pathways. The adjacency matrix defines connectivity, if a connection exists between nodes. A linked matrix K is introduced to indicate whether a follower is directly linked to the leader.

In this framework, the graph describes the directional flow of information, allowing vehicles to dynamically adjust their motion based on received position and velocity data.

$$\begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

Communication Matrix Among the Followers

$$\begin{bmatrix} 1 & 0 & 0 \end{bmatrix}$$

Communication Matrix Between the Leader and Followers

2.2 Electronic Throttle

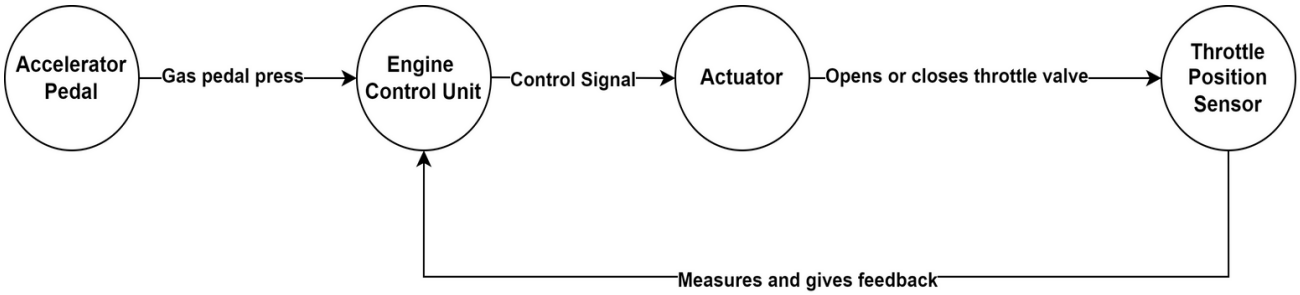


Figure : Electronic Throttle Block Diagram

The electronic throttle (ET) system regulates a vehicle's power by controlling the throttle valve's opening angle, thus adjusting speed and acceleration. In autonomous vehicles (AVs), this control is managed by a microcontroller instead of a human driver.

2.3 Formulation

For vehicle i, the state vector is $s_i = [x_i, v_i, \theta_i]^T$. The predecessor index is j=i−1 (or j=0 for the leader). Useful error signals are

- position gap: $h_i = (x_j - x_i) - \ell$
- spacing error: $e_i = h_i - r$
- velocity error: $\Delta v_i = v_i - v_j$
- throttle mismatch: $\Delta\theta_i = \theta_i - \theta_j - (1/c)(\Delta\dot{v}_i + b\,\Delta v_i)$.

An optimal velocity map $V_{opt}(h_i) = V_1 + V_2 \tanh(C_1 h_i - C_2)$ encourages string-stable behaviour.

Based on the leader follower approach, the dynamics of the leader vehicle can be described as follows:

$$\begin{cases} \dot{x_0}(t) = v_0(t) \\ \dot{v_0}(t) = u_0(t) \end{cases}$$

where $x_0(t) \in R$, $v_0(t) \in R$ represent the position and velocity of the leader. $u_0(t) \in R$ is the input of the leader vehicle.

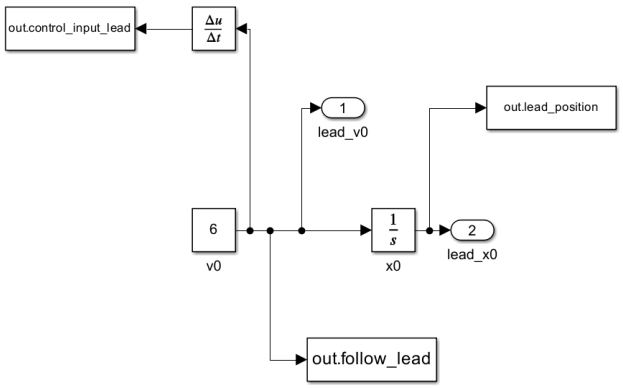


Figure: Leader

The dynamics of the following vehicles can be described as follows:

$$\begin{cases} \dot{x_i}(t) = v_i(t) \\ \dot{v_i}(t) = u_i(t) \end{cases}$$

where $x_i(t) \in R$, $v_i(t) \in R$ represent the position and velocity of the follower. $u_i(t) \in R$ is the input of the follower vehicles.

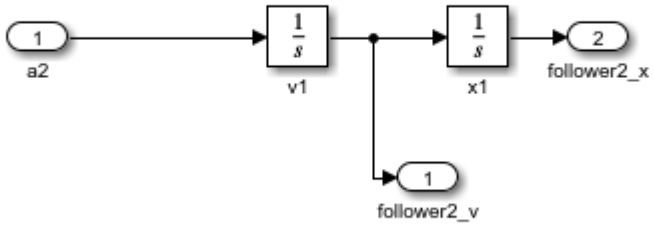


Figure: Follower x and v

3. Platoon Control Protocol

In this section, the platoon control protocol formulated through inter-vehicle position–velocity differences, the electronic throttle angle difference, and a gap-based optimal speed function is detailed, laying the foundation for an integrated control law that enables follower vehicles to maintain safe spacing and velocity matching with both the leader and their neighboring vehicles in real time.

3.1. Position-Velocity Differences Equations

Δx_i and Δv_i are used to compute spacing error and velocity matching between vehicles.

$$\Delta x_i = x_{i-1}(t) - x_i(t)$$

$$\Delta v_i = v_{i-1}(t) - v(t)$$

3.2. Electronic Throttle Angle Difference Equation

b, c are positive control parameters and θ_i captures the ET angle difference based on relative and own velocity.

$$\Delta\theta_i = \theta_{i-1} - \theta_i = \frac{1}{c}(\Delta\dot{v}_i(t) + b\Delta v_i(t))$$

3.3. Optimal Speed Function

- h_i gap between vehicle (i-1) and i.
- V_1 AND V_2 speed scaling constants.
- C_1, C_2 : tanh shape parameters.

$$V(h_i(t)) = V_1 + V_2\tanh(C_1 h_i(t) - C_2)$$

3.4. Control Input Equations

1) Interaction term based on spacing/velocity errors with neighboring vehicles.

2) Error correction relative to the leader vehicle.

3) Speed regulation via the gap-based optimal speed function V_h.

$$\text{Neighbor Interaction Term} = a_{ij}\ [\alpha(\Delta x_{ij}(t) + r_{ij}) + \beta(\Delta v_{ij}(t))]$$

$$\text{Leader-Follower Tracking Term} = -k_i[K(\Delta x_{iL}(t) + r_{ij}) + lamda(\Delta v_{iL}(t))]$$

$$\text{Optimal Speed and Orientation Regulation Term} = \rho[\text{q}(V(h_i(t)) - v_i(t)) + \mu(\Delta v_i(t)) + \gamma(\Delta\theta_i(t)]$$

$$u_i(\text{t}) = -\sum_{j=1}^{n} \text{Neighbor Interaction Term} + \text{Leader}-\text{Follower Tracking Term} + \text{Optimal Speed and Orientation Regulation Term}$$

4. Convergence Analysis

In order to ensure stable platoon formation, it is essential that all follower vehicles eventually reach the same velocity as the leader and maintain the desired spacing. This condition is known as convergence, and it is mathematically analyzed through system stability criteria.

Position and velocity error can be defined as follows:

$$\begin{cases} \dot{x_i}(t) = x_0(t) - x_i(t) - i(r + l) \\ \dot{v_i}(t) = v_0(t) - v_i(t) \end{cases}$$

$x_0(t),\ v_0(t)$: position and velocity of the leader vehicle

$x_i(t),\ v_i(t)$: position and velocity of the i-th follower

r : desired inter-vehicle gap

l : length of each vehicle

These error terms represent the deviation from the ideal platoon configuration.

To ensure convergence, the matrix F must be Hurwitz stable, meaning all its eigenvalues must have negative real parts.

$$F = \begin{matrix} 0_{n*n} & I_{n*n} \\ a*L+B & \beta*L+C \end{matrix}$$

$$B = \begin{bmatrix} p*q*\dot{V} + kappa & 0 & 0 \\ kappa & p*q*\dot{V} & 0 \\ kappa & 0 & p*q*\dot{V} \end{bmatrix} C = \begin{bmatrix} Q + lambda & 0 & 0 \\ p*q + lambda & Q & 0 \\ p*q + lambda & p*q & Q \end{bmatrix}$$

$$L = \begin{bmatrix} \Phi_1 * (a_{11} + a_{12} + a_{13}) & 0 & 0 \\ 0 - 1 & \Phi_2 * (a_{21} + a_{22} + a_{23}) & 0 \\ 0 & 0 - 1 & \Phi_3 * (a_{31} + a_{32} + a_{33}) \end{bmatrix}$$

```
F matrix eigenvalues:
      -0.2937 +          0i
      -1.7092 +          0i
     -0.25145 +     1.3924i
     -0.25145 -     1.3924i
     -0.25145 +     1.3924i
     -0.25145 -     1.3924i

F matrix is Hurwitz Stable: System is stable ✓
F matrix :
            0           0           0           1           0           0
            0           0           0           0           1           0
            0           0           0           0           0           1
     -0.50199           0           0     -2.0029           0           0
          1.5      -2.002           0     -1.5041     -0.5029           0
         -0.5           2      -2.002     -2.0041      0.4959     -0.5029
```

Figure: Matlab Results

5. Numerical Experiments

The simulation runs for 30 s with a sampling interval Δt = 0.05 s for initial states of vehicles are given as follows:

Desired distances set as $r_{1,0} = -5m$, $r_{2,1} = -5m$ *and* $r_{3,2} = -5m$.

Initial positions set as $x_0 = 26m$, $x_1 = 17m$, $x_2 = 10m$ *and* $x_3 = 4m$.

Initial velocities set as $v_0 = \frac{6m}{s}$, $v_1 = \frac{6m}{s}$, $v_2 = \frac{6m}{s}$ *and* $v_4 = 6m/s$.

In this scenario, assume that all vehicles have been equipped with the V2V communication device and established a communication connection.

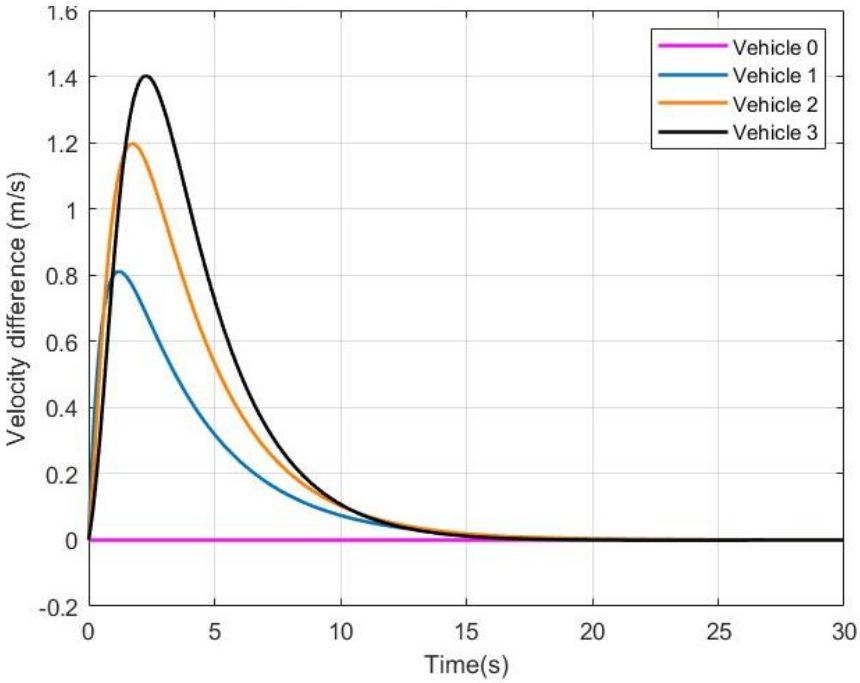


Figure : Velocity difference profile

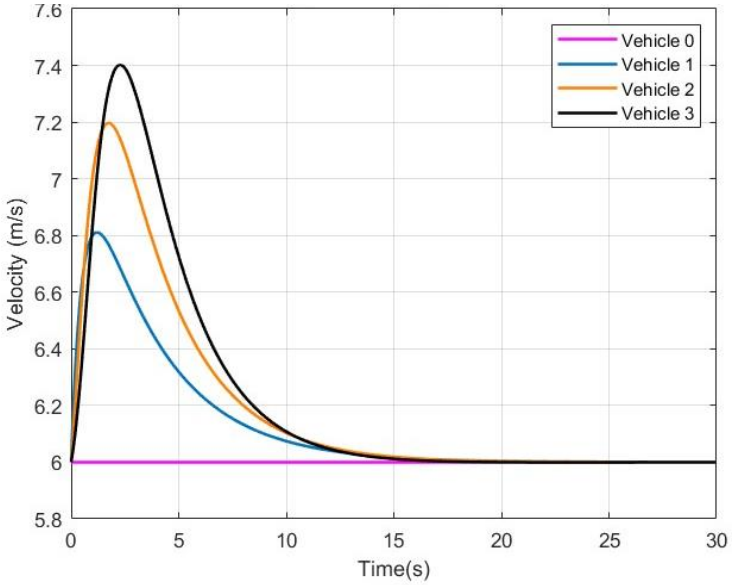


Figure: Velocity profile

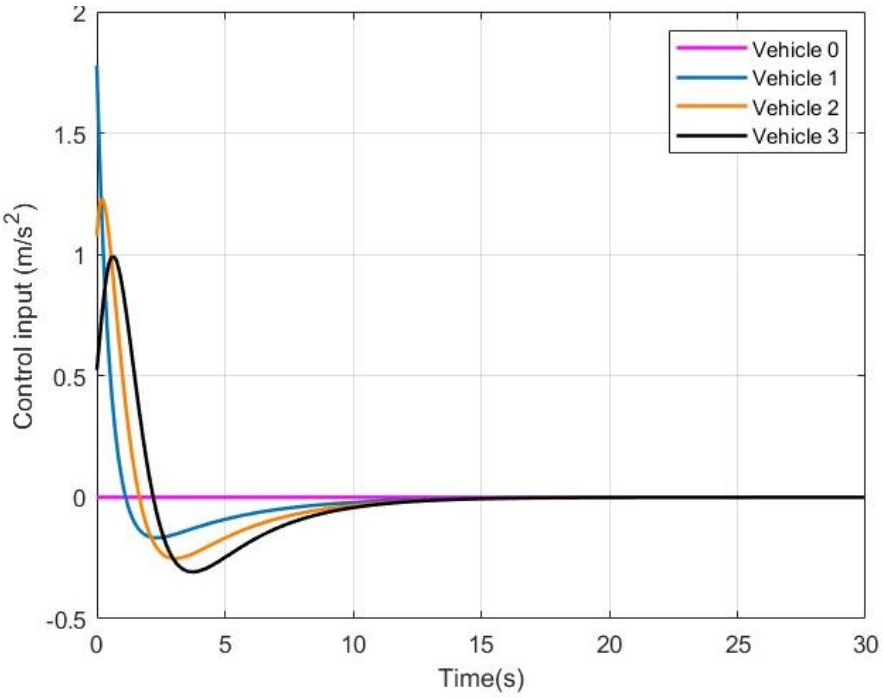


Figure: Control input profile

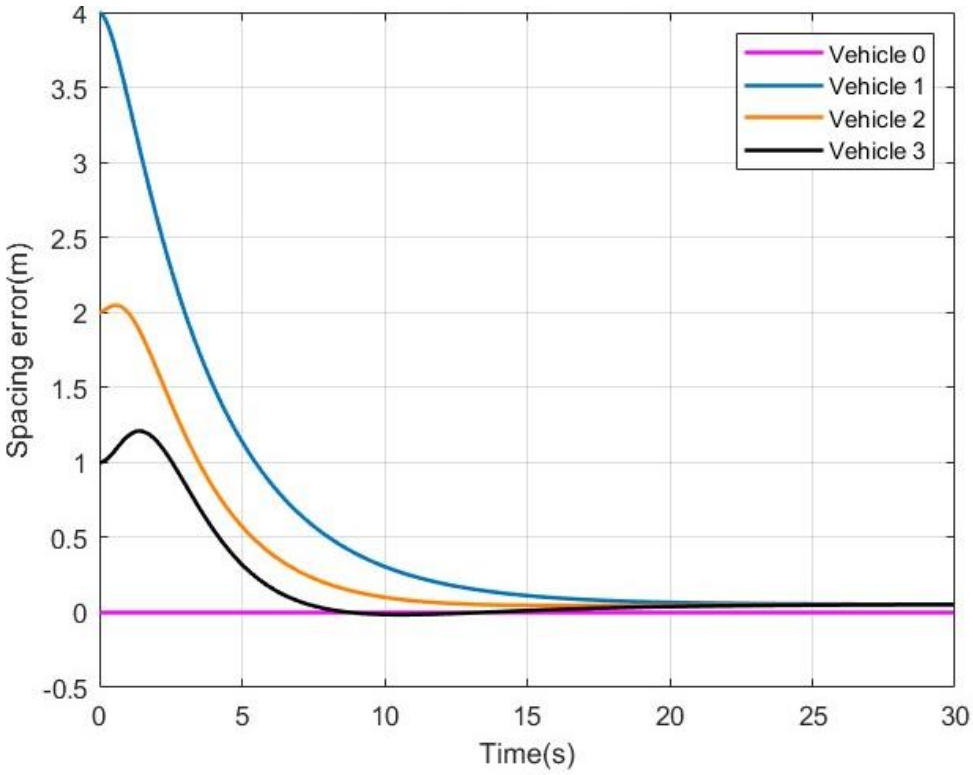


Figure : Spacing error profile

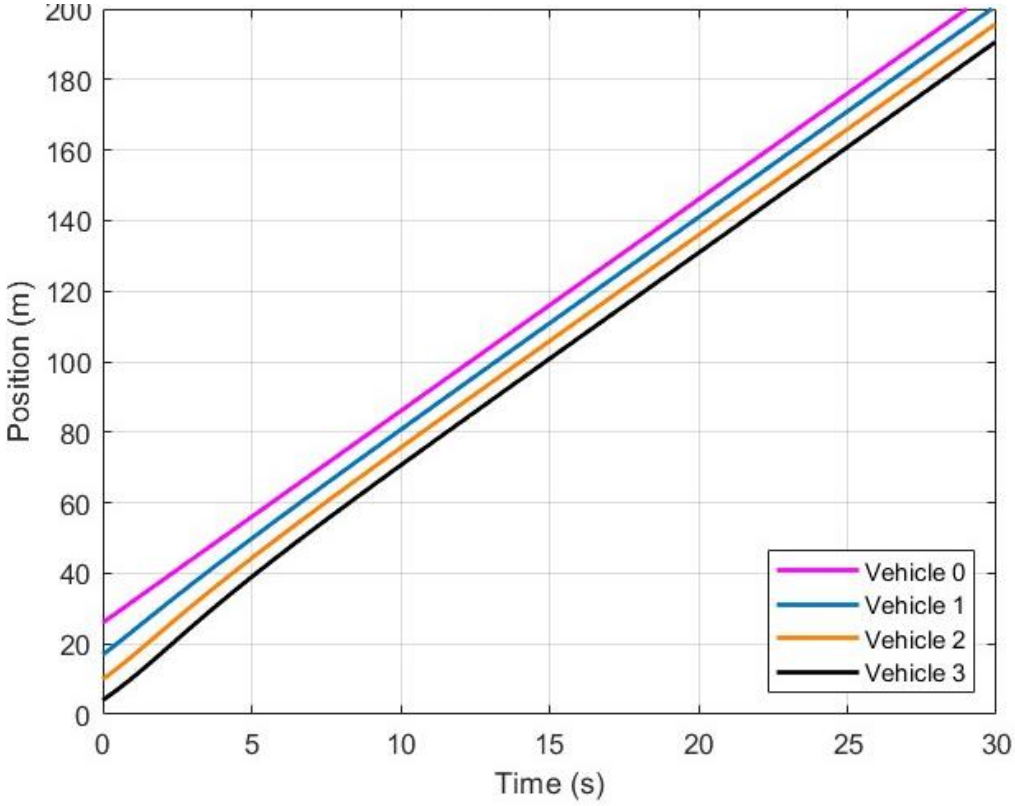


Figure : Position profile

The control input curves indicate that the followers initially apply a positive acceleration, followed by a slight negative input around 3 seconds, and converge to zero by approximately 8 seconds, demonstrating an energy-efficient transient response. As a result, the velocity profiles peak around 7.4 m/s at 4 seconds and then exponentially decay toward the common cruising speed within 15 seconds. The velocity difference plot confirms that the followers reach a maximum overshoot of 1.4 m/s relative to the leader, which is reduced to zero before 12 seconds. The spacing error profiles show that initial deviations of up to 4 meters decay monotonically, ensuring the reestablishment of the desired safe following distance. The position profiles illustrate that all vehicles maintain the desired 5-meter gap in a stable and parallel formation. These results scientifically validate that the proposed platoon control protocol successfully achieves both velocity and spacing consensus with short transient times and minimal control effort.

6. Conclusion

This study proposed a platoon formation control approach that integrates a consensus-based control protocol with the effect of the ET (Electronic Tracking) opening angle under a V2V (vehicle-to-vehicle) communication environment. The stability and consensus conditions of the proposed control algorithm were theoretically validated using the Routh–Hurwitz criterion. Numerical simulations conducted under external disturbances

demonstrated the effectiveness of the proposed control structure in terms of both position and speed tracking.

In future work, the control protocol can be extended to a third-order structure by incorporating the jerk term (the derivative of acceleration), which may enhance control accuracy, ride comfort, and dynamic sensitivity. Additionally, integrating realistic vehicle dynamics—such as tire forces, actuator delays, and steering system behavior—can help bridge the gap between simulation and real-world implementation.

Furthermore, implementing a path-following strategy for the leader vehicle using a Model Predictive Control (MPC) framework would enable it to track reference trajectories effectively. Under such a structure, follower vehicles could maintain platoon coherence through consensus-based protocols leveraging V2V communication. This integrated framework holds strong potential for applications in both autonomous driving and advanced driver-assistance systems (ADAS).

References

[1] Y. Li and C. He, “Connected autonomous vehicle platoon control considering vehicle dynamic information,” in *Proc. 37th Chinese Control Conf. (CCC)*, Wuhan, China, Jul. 25–27, 2018.

[2] S. Guler, M. Menendez, and L. Meier, "Using connected vehicle technology to improve the efficiency of intersections," *Transportation Research Part C: Emerging Technologies*, vol. 46, pp. 121–131, 2014.

[3] Y. Li, L. Zhang, S. Peeta, et al., "A car-following model considering the effect of electronic throttle opening angle under connected environment," *Nonlinear Dynamics*, vol. 85, no. 4, pp. 2115–2125, 2016.

[4] X. Lu, R. Lu, S. Chen, and J. Lu, "Finite-Time Distributed Tracking Control for Multi-Agent Systems With a Virtual Leader," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 60, no. 2, pp. 352–362, 2013.

[5] H. Su, X. Wang, and Z. Lin, "Flocking of multi-agents with a virtual leader Part I: with a minority of informed agents," in *Proceedings of the 46th IEEE Conference on Decision and Control (CDC)*, New Orleans, LA, USA, 2007, pp. 2937–2942.

Appendix: MATLAB Code Implementation

```
% ---------------- Vehicle class ----------------
classdef Vehicle < handle
    properties
        id   (1,1) double {mustBeInteger} = 1    % vehicle ID
        x    (1,1) double = 0                    % position [m]
        v    (1,1) double = 0                    % velocity [m/s]
        theta (1,1) double = 0                   % ET angle [deg]
        a     (1,1) double = 0                   % acceleration [m/s^2]
        L    (1,1) double = 5                    % vehicle length [m]
    end
    methods
        function obj = Vehicle(id, x0, v0, L, theta0)
            if nargin >= 5, obj.theta = theta0; end
            if nargin >= 4, obj.L = L; end
            if nargin >= 3, obj.v = v0; end
            if nargin >= 2, obj.x = x0; end
            obj.id = id;
        end
    end
end

% ---------------- Platoon class ----------------
classdef Platoon < handle
    properties
        veh          Vehicle        % vehicle array (0 = leader)
        A            % adjacency matrix
        k            % leader influence vector
        alpha  = 0.5 ;  beta = 2 ;  kappa = 0.5 ;  lambda = 2 ;
        p      = 0.01;  q = 0.41;  eta = 0.1;  gamma = 0.1;
        c = 1; b = 1; V1 = 6.75; V2 = 7.91; C1 = 0.13; C2 = 1.57;
        l = 5; r = 5; lastU; dt = 0.05;
    end
    methods
        function obj = Platoon(nFollower, x0, v0)
            obj.veh = Vehicle.empty;
            obj.veh(1) = Vehicle(0, x0, v0);
            for i = 1:nFollower
                obj.veh(i+1) = Vehicle(i, x0 - i*(obj.r + obj.l), v0);
            end
            obj.A = diag(ones(nFollower-1,1), -1);
            obj.k = [1, zeros(1, nFollower-1)];
        end

        function step(obj)
            n = numel(obj.veh) - 1;
            x0 = obj.veh(1).x;  v0 = obj.veh(1).v;
            a0 = obj.gamma * obj.veh(1).theta;

            u = zeros(n, 1);
            a_prev = a0;

            for i = 1:n
```

```
                idx = i + 1;
                idxP = (i==1)*1 + (i>1)*i;

                dx = (obj.veh(idx).x - obj.veh(idxP).x) + obj.r;
                dv = obj.veh(idx).v - obj.veh(idxP).v;
                a_i = obj.veh(idx).a;
                ddv = a_prev - a_i;

                dTheta = obj.veh(idx).theta - obj.veh(idxP).theta ...
                       - (1/obj.c)*(ddv + obj.b*dv);

                if i > 1
                    NNterm = obj.alpha*dx + obj.beta*dv;
                else
                    NNterm = 0;
                end

                k_i = obj.k(i);
                LeadTerm = k_i*(obj.kappa*(obj.veh(idx).x - x0 - obj.r) ...
                              + obj.lambda*(obj.veh(idx).v - v0));

                gap = (obj.veh(idxP).x - obj.veh(idx).x) - obj.l;
                Vopt = obj.V1 + obj.V2*tanh(obj.C1*gap - obj.C2);

                Inner = obj.q*(Vopt - obj.veh(idx).v) ...
                      + obj.eta*dv + obj.gamma*dTheta;

                u(i) = -NNterm - LeadTerm + obj.p*Inner;
            end

            obj.lastU = u;

            for i = 1:n
                idx = i+1;
                obj.veh(idx).v = obj.veh(idx).v + u(i)*obj.dt;
                obj.veh(idx).x = obj.veh(idx).x + obj.veh(idx).v*obj.dt;
                obj.veh(idx).theta = obj.veh(idx).theta +
(u(i)/obj.gamma)*obj.dt;
            end

            obj.veh(1).x = obj.veh(1).x + v0*obj.dt;
        end
    end
end
```

SIMULINK VIEW

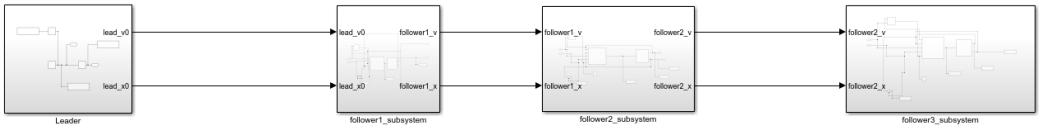


Figure: Overall View

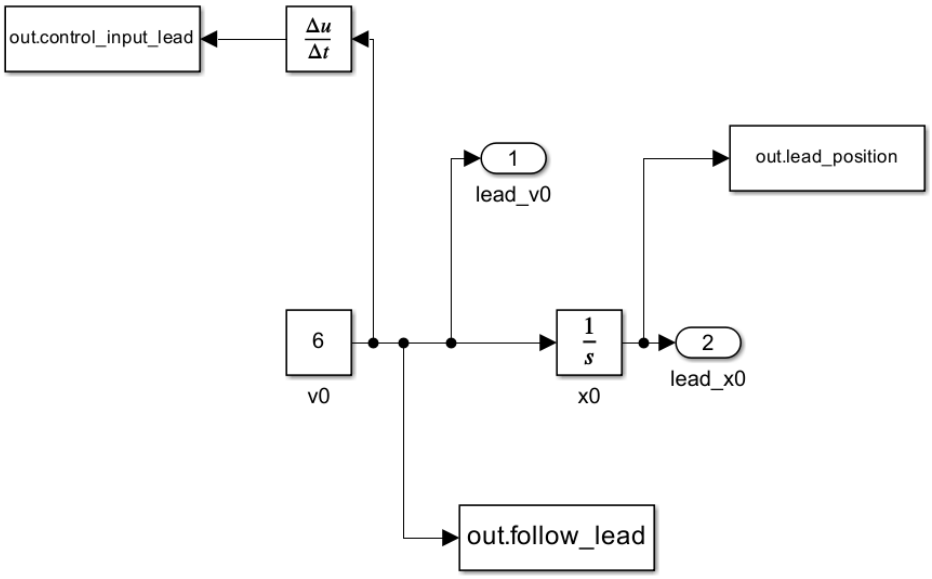


Figure: Leader Subsystem

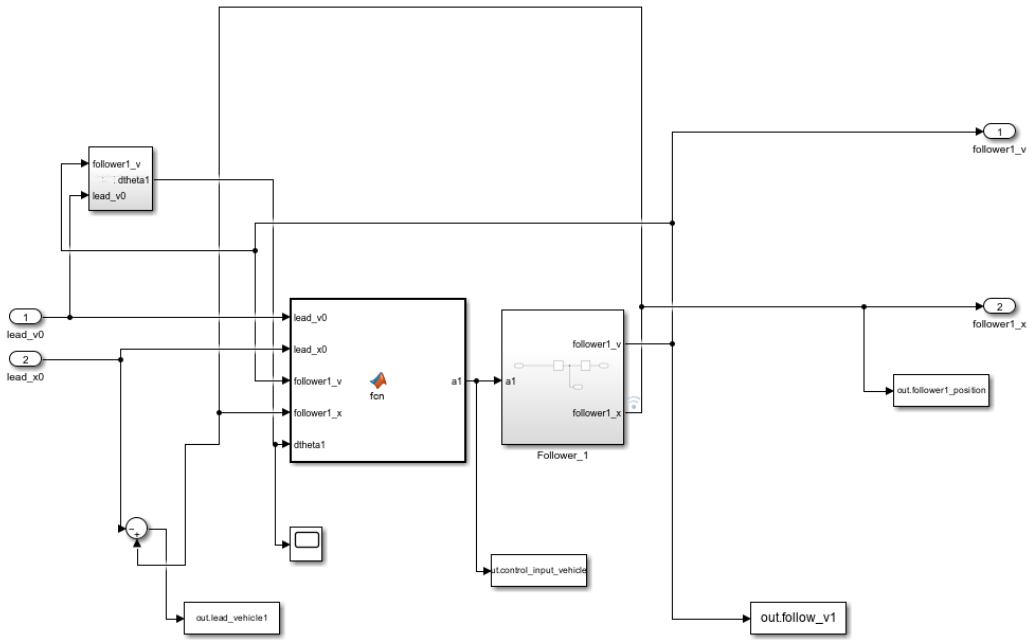


Figure: Follower subsystem (Same for all followers)

Inside code of function block

```
function a1 = fcn(lead_v0, lead_x0, follower1_v, follower1_x, dtheta1)

% Sabit parametreler
alpha = 0.4;
beta = 2;
kappa = 0.5;
lambda = 2;
p = 0.01;
q = 0.41;
eta = 0.05;
gamma = 0.1;

r = 5;  % pozitif takip mesafesi
l = 5;  % araç uzunluğu

% Doğru fark yönü: (ön - kendi - mesafe)
dx = follower1_x - lead_x0 + r;
dv = follower1_v - lead_v0;

% Hedef hız fonksiyonu (ön - kendi - araç boyu)
h = lead_x0 - follower1_x - l;
Vh = 6.75 + 7.91 * tanh(0.13*h - 1.57);

% Kontrol çıktısı (ivme sinyali)
```

```
a1 = -alpha * dx - beta * dv ...
    - kappa * dx - lambda * dv ...
    + p * ( q * (Vh - follower1_v) + eta * (lead_v0 - follower1_v) + gamma *
dtheta1 );

End
```

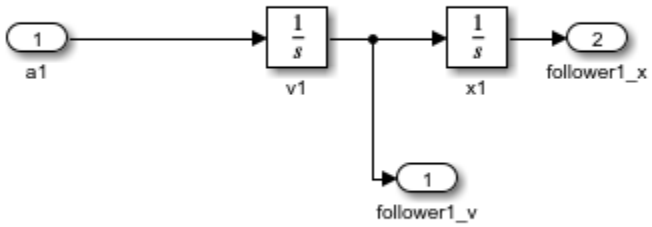


Figure: Follower v and x